

pixel is a triplet corresponding to the red, green and blue components of an RGB image at a specific spatial location.

An RGB image may be viewed as a stack of three gray-scale images that, when fed into the RGB colour monitors, produce a colour image. Eight bits are used to represent the pixel values of each component of the image. Thus an RGB image corresponds to 24 bits.

Detection of the red ball

1. Capture a frame and store it in a variable, say, 'rgb_image'

2. Extract the red, green and blue components of the images and store them in variables fR, fG and fB:

```
fR= rgb_image ( : , : , 1);%extracts  
the red component.
```

```
fG= rgb_image ( : , : , 2);%extracts  
the green component.
```

```
fB= rgb_image ( : , : , 3);%extracts  
the blue component.
```

Here, fR, fG and fB are image matrices. In Matlab, comments are written after '%' sign.

3. Next, find the red object in the image. (R_THRESHOLD=140, (G_THRESHOLD=105 and (B_THRESHOLD=100 are specific numbers called 'threshold.' The technique for finding these numbers is described later on.

The following statement creates a B&W image array 'I':

```
I= ((fR>140) & (fG<105) & (fB<100));
```

That is, the result of logically 'ANDed' image matrices fR, fG and fB is stored in 'I'.

If the following three conditions are satisfied, the pixel value of the image is set to '1':

(i) fR>140 if the value of the red component of the pixel is greater than 140.

(ii) fG<105 if the value of the green component of the pixel is less than 105

(iii) fB<100 if the value of the blue component of the pixel is less than 100

4. Once you make the B&W im-

TABLE II
Instructions for Image Processing

Instruction	Description
im = imread('redball.jpg')	Reads the JPEG image into an image array 'im'
size(im)	Returns the row and column dimensions of an image; for example, [m, n] = size (im);
imshow(im)	Displays the image on the Matlab desktop.
figure, imshow (im2)	Displays image 'im2' in another window; used when needed to display two images in two different windows
imview(im)	Displays the image on the Matlab Desktop. It also shows the pixel value at the corresponding mouse pointer
imwrite(A,filename,fmt)	Writes the image 'A' to the file specified by filename in the format specified by 'fmt'
tf = islogical(A)	Returns logic 1 (true) if 'A' is a logical array and logic '0' (false) otherwise
I = rgb2gray(RGB)	Converts the true-colour RGB image into the grayscale intensity image 'I'
BW = im2bw(I)	Converts the intensity image 'I' into B&W

Note: 1. Syntax of the functions specified can be obtained from the Matlab Help.

2. If the image file is not in the current directory, the whole path of the file should be specified.

3. After every statement there should be a semicolon. Semicolon is given to suppress the output at the command window.

age, you will find that apart from the region of your red ball there are also some unwanted white regions in the image. These unwanted white regions are called 'noise.'

Before you plot the centre of the image, filter the noisy parts of the image as follows:

```
se = strel ('disk', 20); % creates a  
flat, disk-shaped structuring element  
with radius 20  
B= imopen (I, se);%morphological  
opening  
Final= imclose (B, se);%morphological  
closing
```

Morphological opening removes those regions of an object which cannot contain the structuring element, smoothes object contours, breaks thin connections and removes thin protrusions. Morphological closing also tends to smooth the contours of objects besides joining narrow breaks and filling long, thin gulfs and holes smaller than the structuring element.

5. Once you obtain the desired part, find the centre of the ball.

The following statement computes all the connected components in a binary image:

```
[L, n]= bwlabel(Final),
```

Here 'n' is the total number of connected components and 'L' is the label matrix. (Each connected component is given a unique number.)

The following statement:

```
[r, c]= find (L== K) % K= 1, 2 ...n  
returns the row and column indices for  
all pixels belonging to the Kth object:  
rbar= mean(n);  
cbar= mean(c);
```

Variables 'rbar' and 'cbar' are the coordinates of the centre of mass.

As you have already filtered the image, the final image contains only one white region. But in case there is a computational fault due to excessive noise, you might have two connected components. So form a loop from '1' to 'n' using 'for' statement, thus calculating the centre of mass for all objects. The syntax is:

```
for K=1:n
```

If there are no components in a frame, the control doesn't enter the loop and 'rbar' and 'cbar' remain initialised to zero.

For checking the output on the computer, use the following instructions:

```
imshow (rgb_image);  
hold on  
plot (cbar, rbar, 'marker', 's', 'r',
```